Caching Strategies for High-Traffic Web Services

A practical engineering guide

1. Why cache, and how to read and write through a cache

A cache stores copies of data in a location that is faster to read than the original source. In a web service, the source of truth is usually a database, and the cache is an in-memory store such as Redis or Memcached, or a local in-process map. Reading from memory on the same machine takes well under a microsecond, a round trip to a cache server in the same data centre typically takes a fraction of a millisecond, and a database query that touches disk can take several milliseconds or more. Caching is therefore one of the most effective ways to reduce latency and to protect a database from load.

The most important number for a cache is its hit ratio: the fraction of requests that are served from the cache rather than from the source of truth. A cache with a 50 percent hit ratio halves the read load on the database; a cache with a 99 percent hit ratio reduces it a hundredfold. Small changes in a high hit ratio therefore have a large effect on the backend.

Read patterns

- Cache-aside, also called lazy loading, is the most common pattern. The application first looks in the cache. On a miss it reads from the database, stores the result in the cache and returns it. Only data that is actually requested ends up in the cache.
- Read-through caching moves the same logic into the cache layer: the application always asks the cache, and the cache itself loads missing entries from the database.

Write patterns

- With write-through caching, every write goes to the cache and to the database as part of the same operation. Cached data stays fresh, at the cost of higher write latency.
- With write-behind caching, also called write-back caching, writes go to the cache first and are flushed to the database asynchronously, often in batches. Write latency is low, but writes that have not yet been flushed can be lost if the cache fails.
- With write-around caching, writes go directly to the database and the cache is filled only when the data is next read. This avoids filling the cache with data that is written once and rarely read.

2. Invalidation and consistency

As the saying attributed to Phil Karlton goes, there are only two hard things in computer science: cache invalidation and naming things. Any cache can return data that is older than the source of truth. The design question is how stale the data may be and for how long.

Expiration with a time to live

The simplest tool is a time to live (TTL). Each entry expires after a fixed period, after which the next read loads a fresh copy. A short TTL bounds staleness but lowers the hit ratio; a long TTL raises the hit ratio but allows older data to be served. A TTL is a safety net even when other invalidation methods are used, because it guarantees that a missed invalidation cannot keep a stale value alive forever.

Explicit invalidation

For data that must reflect writes quickly, the application invalidates the cache when the underlying data changes. Deleting the cached entry on write is usually safer than updating it in place. If two concurrent writers both update the cache, the cache may end up holding the value from the write that reached the database first, even though the database holds the later value.

Cache-aside with delete-on-write still has a race. A reader misses, reads the old value from the database, and is delayed. Meanwhile a writer updates the database and deletes the cache entry. The delayed reader then stores the old value in the cache, where it stays until the TTL expires. Common mitigations are short TTLs, deleting the entry a second time after a short delay, and versioned keys, in which the cache key includes a version number that changes on every write so that an old value is stored under a key nobody reads.

Choosing what to cache

Good candidates are read-heavy data that is expensive to compute and tolerant of slight staleness, such as product catalogues, rendered page fragments and aggregated dashboard results. Poor candidates are data that changes on almost every read, data that requires strict consistency, such as account balances during a payment, and highly personalised data with little reuse between requests.

3. Scaling caches across many machines

A single cache server eventually runs out of memory or network capacity, so large services spread their cache across a cluster. The client must then decide which server stores each key.

Consistent hashing

The naive approach computes the hash of a key modulo the number of servers. When a server is added or removed, almost every key maps to a different server, and the cluster suddenly behaves like an empty cache. Consistent hashing, introduced by David Karger and colleagues in 1997, places both servers and keys on a hash ring and assigns each key to the next server clockwise on the ring. When a server is added to a cluster of n servers, only about 1/n of the keys move to the new server, and all other keys stay where they were. Implementations place each physical server on the ring many times as virtual nodes, so that load is spread evenly and a failed server's keys are shared among many survivors.

Hot keys

Hashing spreads keys evenly, but not requests. A single very popular key, such as the home page of a large event, can overload the one server that stores it. Common remedies are replicating hot keys to several servers, adding a small local in-process cache in front of the distributed cache, and splitting one logical key into several physical copies that are read at random.

Cache stampedes

A cache stampede, also called the thundering herd problem, happens when a popular entry expires or the cache is emptied and many concurrent requests miss at the same moment. Every one of them queries the database for the same value, and the sudden load can overwhelm it. Restarting a cache cluster, which empties it, can trigger a stampede across many keys at once.

Several techniques prevent stampedes:

- Request coalescing ensures that only one request recomputes a missing value while the others wait for that result. This is often implemented with a short-lived lock in the cache.
- Adding random jitter to TTLs spreads expirations over time, so that entries written together do not all expire together.
- Probabilistic early expiration, such as the XFetch algorithm described by Vattani, Chierichetti and Lowenstein in 2015, lets individual requests refresh an entry slightly before it expires, with a probability that rises as expiry approaches.
- Serving stale data while revalidating keeps returning the old value for a short grace period while a single background request loads the new value.

4. Eviction, sizing and monitoring

When a cache is full, it must evict an entry to make room for a new one. The eviction policy decides which entry to remove.

- Least recently used (LRU) evicts the entry that has not been accessed for the longest time. It adapts well to changing access patterns and is the most widely used policy.
- Least frequently used (LFU) evicts the entry with the fewest accesses. It protects popular entries but reacts slowly when popularity changes.
- First in, first out (FIFO) evicts the oldest entry regardless of how it is used. It is cheap but usually gives a lower hit ratio than LRU.
- W-TinyLFU, used by the Caffeine library for Java, combines a small admission window with a compact frequency sketch. A new entry is admitted only if it is estimated to be accessed more often than the entry it would replace, which keeps one-off requests from pushing out popular data.

Sizing

A cache only needs to hold the working set: the data that is actually requested within the typical lifetime of an entry. Access patterns in web services are usually highly skewed, so a cache that holds a small fraction of the full data set can still achieve a high hit ratio. The way to size a cache is to measure the hit ratio at several sizes, or to simulate the eviction policy against a recorded trace of real requests, rather than to guess.

What to monitor

A production cache should be monitored with at least the following metrics:

- the hit ratio, overall and for the most important key prefixes;
- the eviction rate, because a rising eviction rate is an early sign that the cache is too small;
- latency percentiles such as p50 and p99 for cache operations;
- memory usage and fragmentation on each cache server;
- the load on the database behind the cache, which reveals how much work the cache is really absorbing.

Checklist

- Use cache-aside unless there is a specific reason to choose another pattern.
- Always set a TTL, even when entries are invalidated explicitly.
- Delete entries on write rather than updating them in place.
- Protect expensive keys against stampedes with request coalescing and TTL jitter.
- Use consistent hashing with virtual nodes for distributed caches.
- Treat the cache as an optimisation: the service must remain correct, if slower, when the cache is empty.